

Experiment No. 8

Experiment Name - Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

✓ Steps

✓ NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Import the required libraries and load a real-world video sequence or capture live video using a webcam.

```
import cv2
import numpy as np
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode

display(Javascript("""
async function recordVideo() {
  const video = document.createElement('video');
  video.width = 640;
  video.height = 480;
  video.autoplay = true;

  const stream = await navigator.mediaDevices.getUserMedia({
    video: true,
    audio: false
  });

  document.body.appendChild(video);
  video.srcObject = stream;

  const recorder = new MediaRecorder(stream);
  let chunks = [];

  recorder.ondataavailable = e => chunks.push(e.data);

  recorder.start();

  await new Promise(resolve => setTimeout(resolve, 10000));

  recorder.stop();

  await new Promise(resolve => recorder.onstop = resolve);
})
```

```

        stream.getTracks().forEach(track => track.stop());
        video.remove();

        const blob = new Blob(chunks, {type: 'video/webm'});
        const reader = new FileReader();

        return await new Promise(resolve => {
            reader.onloadend = () => resolve(reader.result);
            reader.readAsDataURL(blob);
        });
    }
    """))

data = eval_js("recordVideo()")

video_data = b64decode(data.split(',')[1])

with open("webcam_video.webm", "wb") as f:
    f.write(video_data)

print("Live video recorded successfully.")
print("Saved as: webcam_video.webm")

```

```

Live video recorded successfully.
Saved as: webcam_video.webm

```

2. Identify a moving object or select feature points using the Shi-Tomasi Corner Detection algorithm.

```

# Shi-Tomasi Corner Detection
import cv2
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, frame = cap.read()

if ret:
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    corners = cv2.goodFeaturesToTrack(
        gray,
        maxCorners=100,
        qualityLevel=0.3,
        minDistance=7
    )

    corners = corners.astype(int)

    for corner in corners:
        x, y = corner.ravel()
        cv2.circle(frame, (x, y), 4, (0, 255, 0), -1)

    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.title("Shi-Tomasi Feature Points")
    plt.axis("off")
    plt.show()

```

```
cap.release()
```

Shi-Tomasi Feature Points



3. Apply the Lucas-Kanade Optical Flow algorithm to track the movement of the selected object across consecutive video frames.

```
# Lucas-Kanade Optical Flow
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS |
              cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

mask = np.zeros_like(old_frame)
```

```

while True:
    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is None:
        break

    good_new = p1[st == 1]
    good_old = p0[st == 1]

    for new, old in zip(good_new, good_old):
        x1, y1 = new.ravel()
        x2, y2 = old.ravel()

        cv2.line(
            mask,
            (int(x2), int(y2)),
            (int(x1), int(y1)),
            (0, 255, 0),
            2
        )

        cv2.circle(
            frame,
            (int(x1), int(y1)),
            4,
            (0, 0, 255),
            -1
        )

    result = cv2.add(frame, mask)

    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)

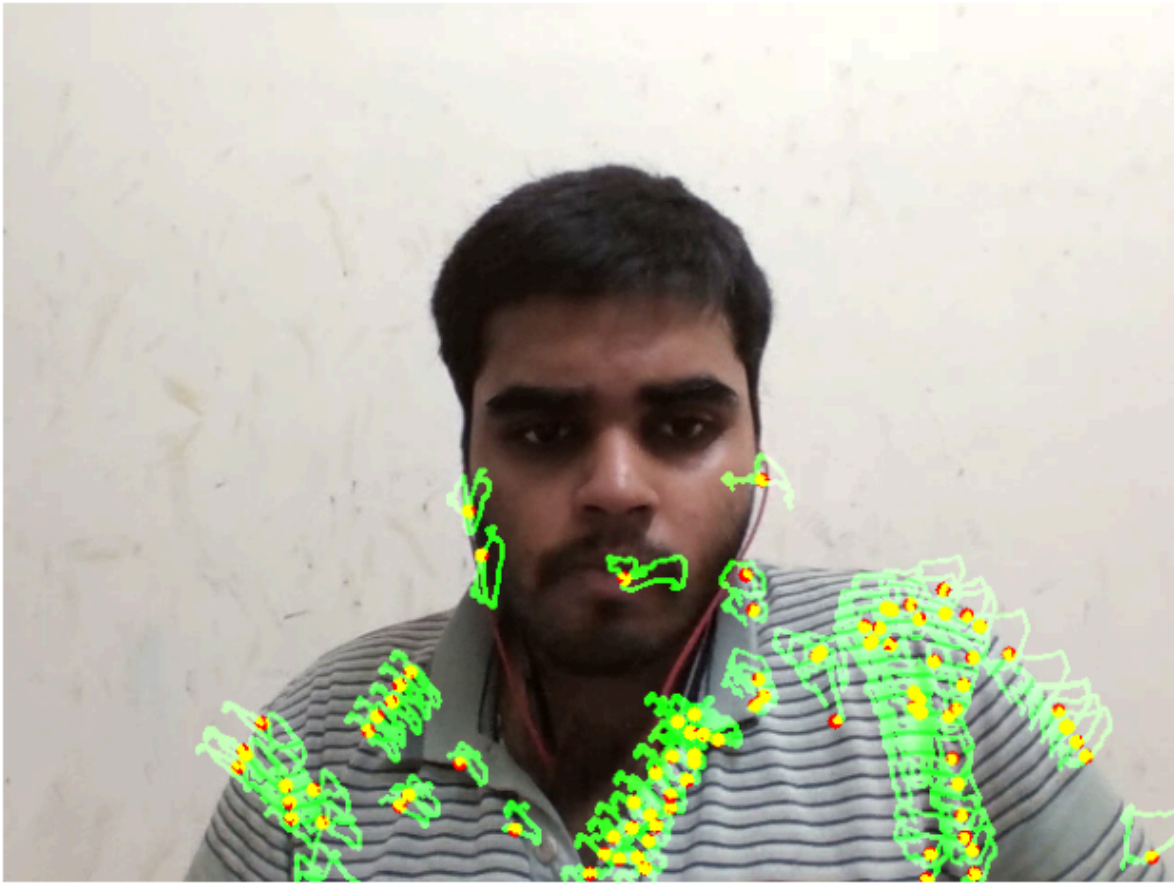
last_frame = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

cap.release()

plt.figure(figsize=(10, 6))
plt.imshow(last_frame)
plt.title("Lucas-Kanade Optical Flow Tracking")
plt.axis("off")
plt.show()

```

Lucas-Kanade Optical Flow Tracking



4. Draw motion trajectories and displacement vectors to visualize the path followed by the moving object.

```
# Draw Motion Trajectories and Displacement Vectors
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

mask = np.zeros_like(old_frame)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS |
              cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

while True:
    ret, frame = cap.read()
```

```

        if not ret:
            break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is None:
        break

    good_new = p1[st == 1]
    good_old = p0[st == 1]

    for new, old in zip(good_new, good_old):

        x1, y1 = new.ravel()
        x2, y2 = old.ravel()

        cv2.line(
            mask,
            (int(x2), int(y2)),
            (int(x1), int(y1)),
            (0, 255, 0),
            2
        )

        cv2.arrowedLine(
            frame,
            (int(x2), int(y2)),
            (int(x1), int(y1)),
            (255, 0, 0),
            2,
            tipLength=0.3
        )

        cv2.circle(
            frame,
            (int(x1), int(y1)),
            4,
            (0, 0, 255),
            -1
        )

    result = cv2.add(frame, mask)

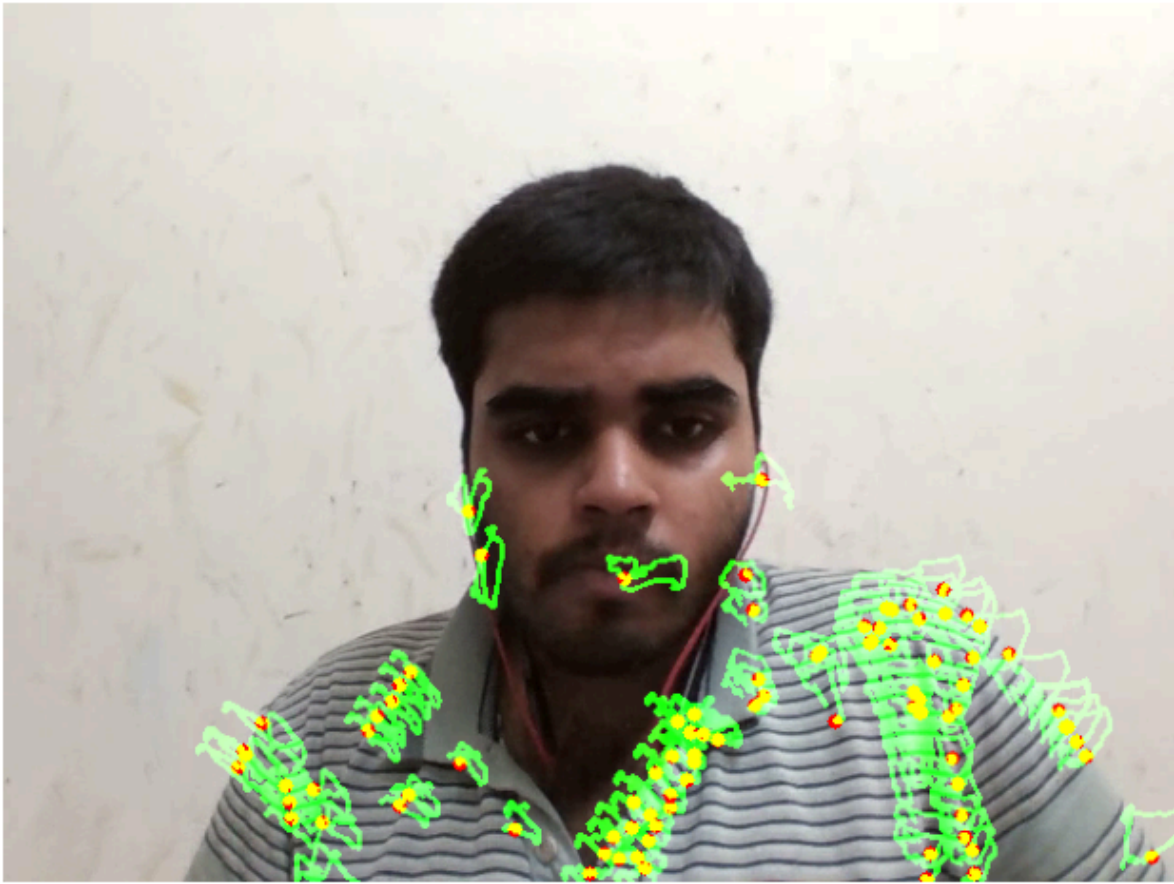
    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)

cap.release()

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Motion Trajectories and Displacement Vectors")
plt.axis("off")
plt.show()

```

Motion Trajectories and Displacement Vectors



5. Calculate the direction and magnitude of object motion and display the tracking results in real time.

```
# Calculate Direction and Magnitude of Motion
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS |
              cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

last_frame = None

while True:
    ret, frame = cap.read()
```

```

if not ret:
    break

frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

p1, st, err = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    p0,
    None,
    **lk_params
)

if p1 is None:
    break

good_new = p1[st == 1]
good_old = p0[st == 1]

if len(good_new) == 0:
    break

for new, old in zip(good_new, good_old):

    x1, y1 = new.ravel()
    x2, y2 = old.ravel()

    dx = x1 - x2
    dy = y1 - y2

    magnitude = math.sqrt(dx**2 + dy**2)
    direction = math.degrees(math.atan2(dy, dx))

    cv2.arrowsLine(
        frame,
        (int(x2), int(y2)),
        (int(x1), int(y1)),
        (255, 0, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        frame,
        (int(x1), int(y1)),
        4,
        (0, 0, 255),
        -1
    )

    cv2.putText(
        frame,
        f"Mag: {magnitude:.2f}px Dir: {direction:.1f}°",
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 0),
        2
    )

old_gray = frame_gray.copy()

```

```

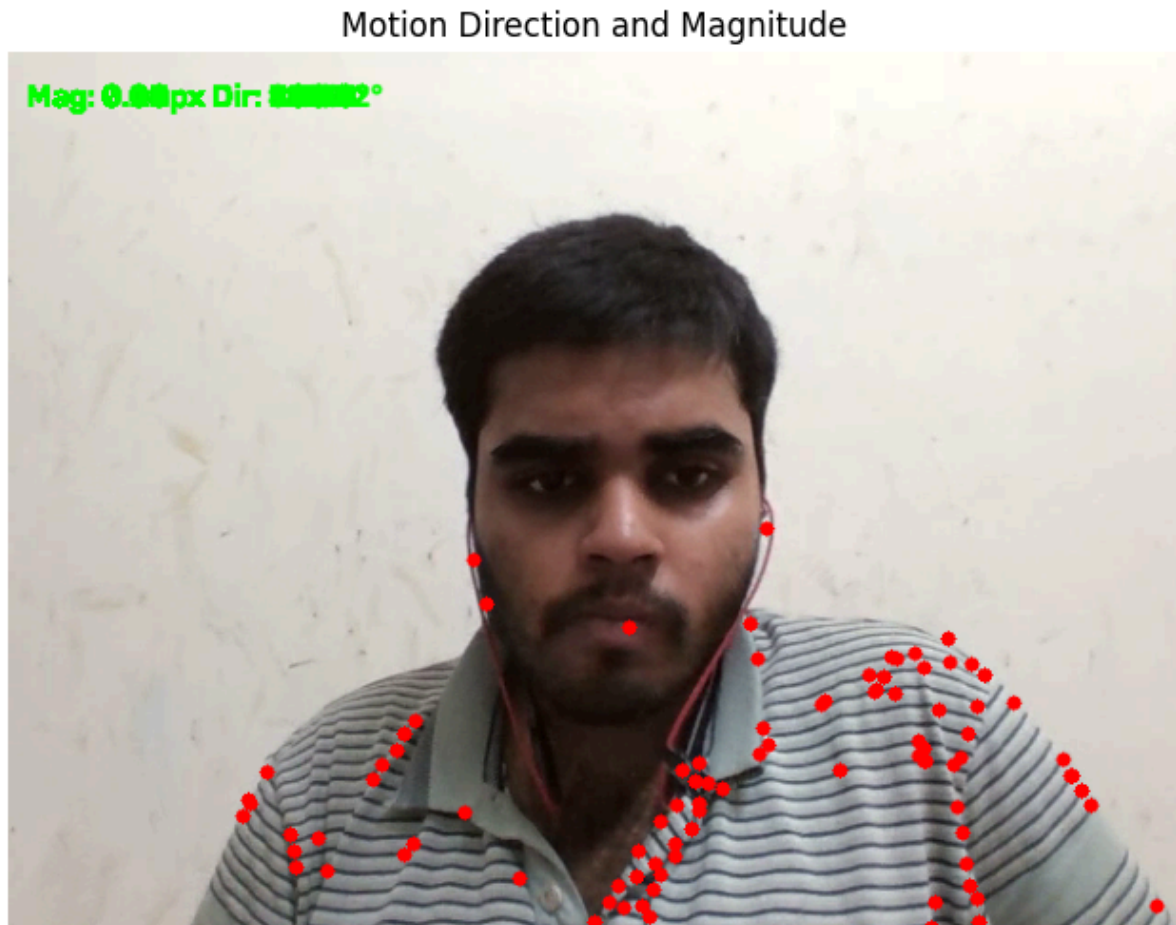
p0 = good_new.reshape(-1, 1, 2)

last_frame = frame.copy()

cap.release()

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(last_frame, cv2.COLOR_BGR2RGB))
plt.title("Motion Direction and Magnitude")
plt.axis("off")
plt.show()

```



6. Repeat the experiment using the Farneback Dense Optical Flow algorithm and compare the tracking performance.

```

# Farneback Dense Optical Flow
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

last_frame = None

while True:
    ret, frame = cap.read()

    if not ret:
        break

```

```

frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

step = 20

for y in range(0, frame.shape[0], step):
    for x in range(0, frame.shape[1], step):

        dx = flow[y, x, 0]
        dy = flow[y, x, 1]

        cv2.arrowsLine(
            frame,
            (x, y),
            (int(x + dx), int(y + dy)),
            (0, 255, 0),
            1,
            tipLength=0.3
        )

avg_magnitude = np.mean(magnitude)

cv2.putText(
    frame,
    f"Average Motion: {avg_magnitude:.2f} pixels",
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 255),
    2
)

last_frame = frame.copy()
old_gray = frame_gray.copy()

cap.release()

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(last_frame, cv2.COLOR_BGR2RGB))
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()

```

Farneback Dense Optical Flow



7. Evaluate the impact of object speed, illumination changes, occlusion, and camera movement on tracking accuracy.

```
# Evaluate Impact of Speed, Illumination, Occlusion and Camera Movement
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS |
              cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

motion_values = []
frame_count = 0

while True:
    ret, frame = cap.read()
```

```

        if not ret:
            break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is None:
        break

    good_new = p1[st == 1]
    good_old = p0[st == 1]

    if len(good_new) == 0:
        break

    displacement = np.linalg.norm(
        good_new - good_old,
        axis=1
    )

    average_motion = np.mean(displacement)
    motion_values.append(average_motion)

    frame_count += 1

    cv2.putText(
        frame,
        f"Average Motion: {average_motion:.2f} pixels",
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 0),
        2
    )

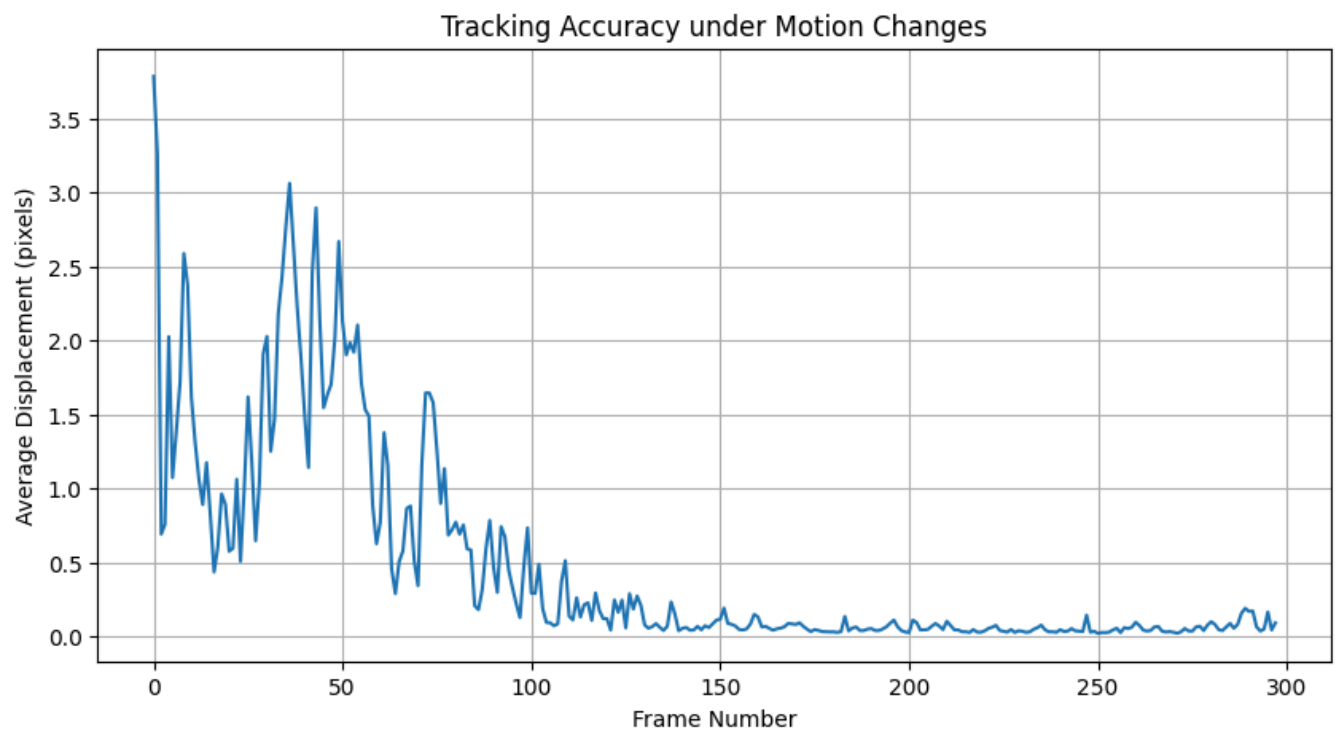
    last_frame = frame.copy()

    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)

cap.release()

# Plot tracking motion over time
plt.figure(figsize=(10, 5))
plt.plot(motion_values)
plt.xlabel("Frame Number")
plt.ylabel("Average Displacement (pixels)")
plt.title("Tracking Accuracy under Motion Changes")
plt.grid()
plt.show()

```



✓ Evaluation:

1. Higher object speed produces larger displacement.
2. Illumination changes may reduce feature tracking accuracy.
3. Occlusion can cause loss of tracked feature points.
4. Camera movement produces motion across many image points.
5. Stable illumination and limited camera movement improve tracking accuracy.

8. Analyze the tracked motion to estimate parameters such as object displacement, trajectory, and average speed.

```
# Analyze Displacement, Trajectory and Average Speed
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("webcam_video.webm")

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
```

```

        criteria=(cv2.TERM_CRITERIA_EPS |
                  cv2.TERM_CRITERIA_COUNT, 10, 0.03)
    )

    total_displacement = 0
    motion_values = []
    trajectory = []

    fps = cap.get(cv2.CAP_PROP_FPS)

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is None:
            break

        good_new = p1[st == 1]
        good_old = p0[st == 1]

        if len(good_new) == 0:
            break

        displacement = np.linalg.norm(
            good_new - good_old,
            axis=1
        )

        avg_displacement = np.mean(displacement)

        total_displacement += avg_displacement
        motion_values.append(avg_displacement)

        center = np.mean(good_new, axis=0)
        trajectory.append(center)

        old_gray = frame_gray.copy()
        p0 = good_new.reshape(-1, 1, 2)

    cap.release()

    total_time = len(motion_values) / fps

    average_speed = total_displacement / total_time

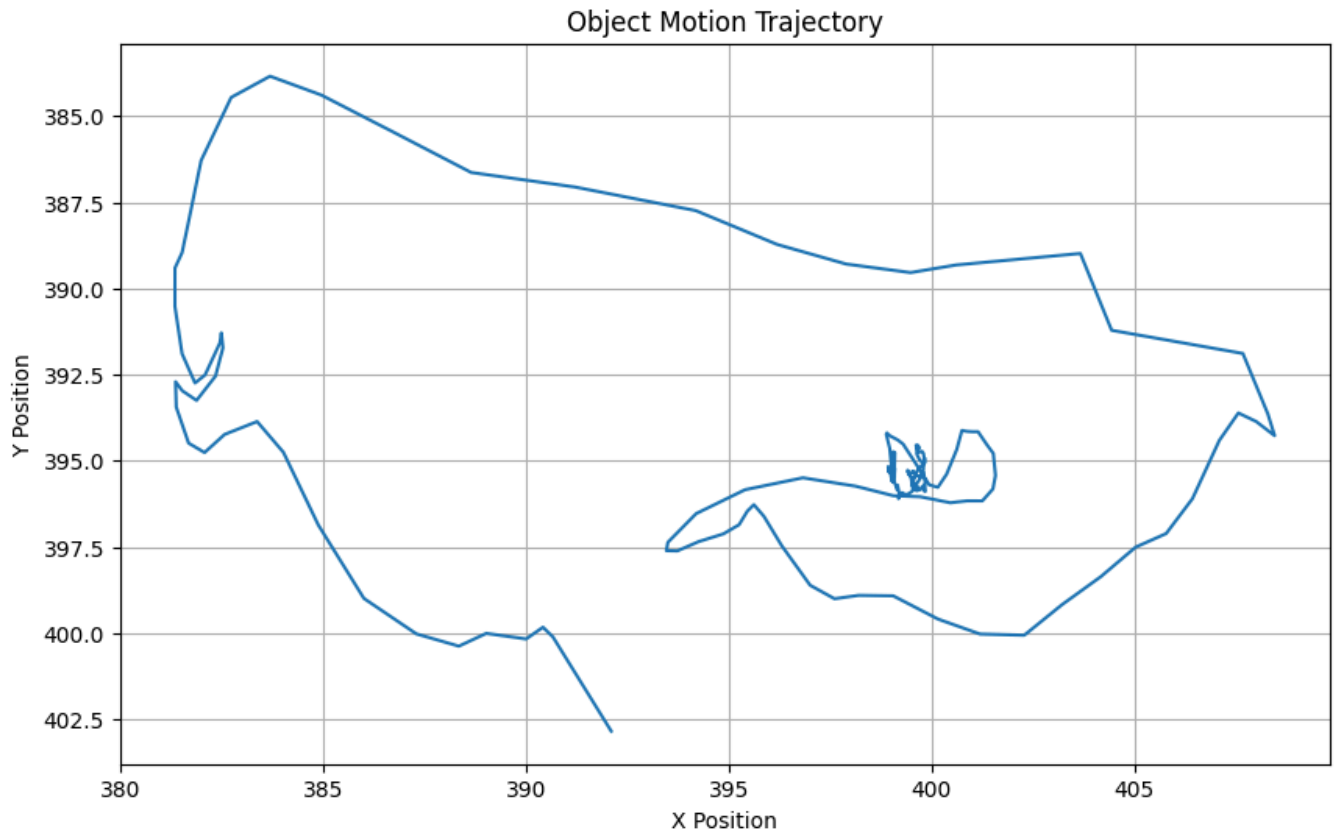
    print(f"Total Displacement: {total_displacement:.2f} pixels")
    print(f"Average Speed: {average_speed:.2f} pixels/second")

    # Plot trajectory
    trajectory = np.array(trajectory)

```

```
plt.figure(figsize=(10, 6))
plt.plot(trajecory[:, 0], trajecory[:, 1])
plt.gca().invert_yaxis()
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.title("Object Motion Trajectory")
plt.grid()
plt.show()
```

Total Displacement: 142.91 pixels
Average Speed: 479.58 pixels/second



9. Compare the effectiveness of optical flow-based tracking with conventional frame-by-frame object detection techniques.

Comparison: Optical Flow vs Conventional Object Detection

Parameter	Optical Flow Tracking	Frame-by-Frame Object Detection
Working	Tracks motion of features between consecutive frames	Detects the object independently in each frame
Speed	Generally faster after initialization	Usually slower because detection runs on every frame
Motion Information	Provides direction, displacement and trajectory	Mainly provides object location
Computation	Requires less computation for feature tracking	Requires more computation for repeated detection
Occlusion	Tracking may fail when features are hidden	Can detect the object again after it becomes visible
Camera Movement	Sensitive to camera motion	Can handle camera movement if the detector is robust
Accuracy	Depends on good feature points and motion	Depends on the object detection model

10. Document the observations and discuss practical applications of optical flow in intelligent vision systems.

Observations

1. Optical flow successfully detects and tracks the motion of objects between consecutive video frames.
2. Lucas-Kanade Optical Flow tracks selected feature points and provides their direction and displacement.
3. Farneback Dense Optical Flow provides motion information for a large number of pixels in the image.
4. Tracking accuracy can be affected by object speed, illumination changes, occlusion, and camera movement.
5. Optical flow is useful for estimating object displacement, trajectory, direction, and average speed.

Practical Applications

- **Traffic Monitoring:** Tracking vehicles and analyzing traffic flow.
- **Human Activity Recognition:** Detecting and analyzing human movements.
- **Autonomous Vehicles:** Estimating motion of vehicles, pedestrians, and obstacles.
- **Video Surveillance:** Detecting unusual movements and activities.
- **Sports Analytics:** Tracking player and ball movements.
- **Robotics:** Helping robots understand motion and navigate their surroundings.
- **Intelligent Vision Systems:** Supporting real-time motion detection, tracking, and scene understanding.

Questions and Answers

1. How does object tracking differ from object detection?

Object detection identifies and locates objects in an individual frame, while **object tracking** follows the same object across multiple consecutive frames. Detection answers "Where is the object?", whereas tracking also maintains its movement over time.

2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the movement of pixels or feature points between consecutive video frames. By tracking these changes continuously, the system can determine the object's **direction, displacement, trajectory, and speed** in real time.

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi Corner Detection identifies strong and distinctive feature points in the first frame. The **Lucas-Kanade algorithm** then tracks these points in subsequent frames to estimate their movement.

4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow is suitable because it estimates the **direction and magnitude of motion** between consecutive frames. It provides useful information about object movement, trajectories, and speed without requiring complete object detection in every frame.

5. What challenges arise while tracking fast-moving or partially occluded objects?

Fast-moving objects can move significantly between frames, making feature matching difficult. Partial occlusion can hide important feature points, causing the tracker to lose the object. Changes in illumination, blur, and camera movement can also reduce tracking accuracy.

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow mainly uses pixel or feature-point motion and is generally computationally lighter. Deep learning-based tracking uses trained neural networks to recognize and track objects and can provide stronger object identification, but usually requires more computational resources and training data.

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

In **traffic monitoring**, Optical Flow can estimate vehicle movement, speed, direction, and traffic flow. In **autonomous driving**, it can help estimate the motion of vehicles, pedestrians, and obstacles and support navigation and collision-avoidance systems.

8. Explain the effect of camera motion on Optical Flow estimation.

Camera movement causes many or most pixels in the scene to appear moving even when the objects themselves are stationary. This can make it difficult to distinguish **object motion from camera motion**. Camera stabilization or background-motion estimation can help reduce this problem.

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

1. Traffic monitoring and vehicle tracking
2. Autonomous vehicles
3. Video surveillance
4. Human activity recognition
5. Sports analytics

10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

Optical Flow provides information about **motion direction, speed, and trajectory**. In surveillance, it helps detect unusual movements; in robotics, it supports navigation and obstacle avoidance; and in human activity recognition, it helps identify activities based on patterns of body movement.